

P2 - DÉBUGGEZ UNE APPLICATION JAVA

1. Présentation de l'entreprise
2. Objectifs du projet
3. Présentation de l'application
4. Problèmes identifiés dans le code initial
5. Solutions mises en place
6. Architecture mise en place
7. Principe d'injection de dépendances
8. Difficultés rencontrées
9. Solutions apportées
10. Améliorations possibles
11. Conclusion

PRÉSENTATION DE L'ENTREPRISE

Heme Biotech est une entreprise de petite taille, réputée dans le secteur pharmaceutique. Leur spécialité est le traitement des troubles sanguins chez l'humain et l'animal.

OBJECTIFS DU PROJET

1. Comprendre le langage de programmation Java
2. Construire un projet de code collaboratif
 - Mise en place d'un versionning du code avec GIT
 - Identifier et corriger les bugs
 - Passer d'un couplage fort à un couplage faible
 - Décomposer les fonctionnalités en plusieurs méthodes et classes pour améliorer la maintenabilité

PRÉSENTATION DE L'APPLICATION

Un programme d'analyse des tendances.

1. Lire une liste de symptômes depuis un fichier texte
2. Compter le nombre d'occurrences de chaque symptôme
3. Trier les résultats par ordre alphabétique
4. Écrire les résultats dans un fichier de sortie

PRÉSENTATION DE L'APPLICATION

Exemple d'entrée

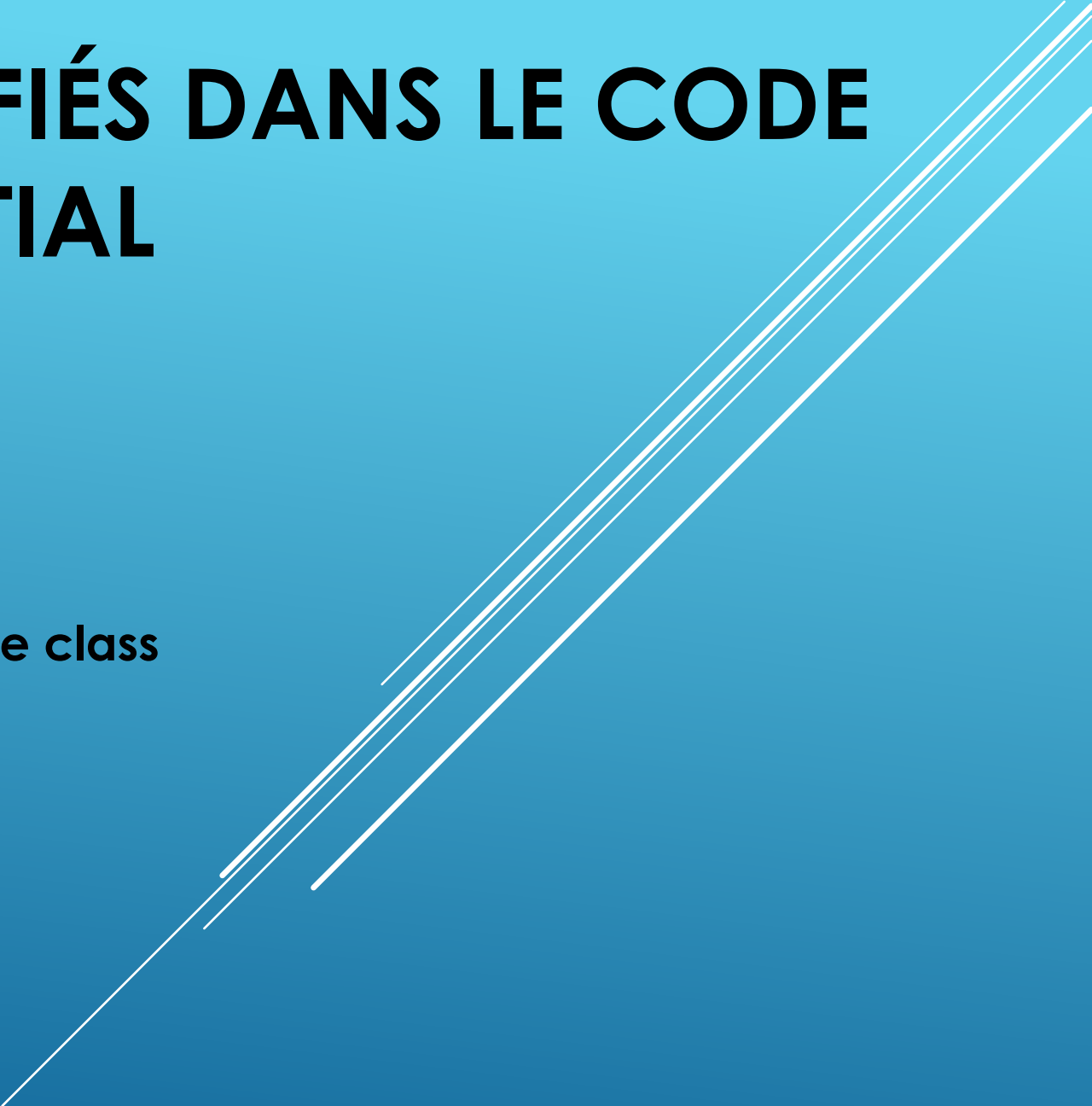
- Fichier symptoms.txt
- headache
- rash
- headache
- dizziness
- rash

PRÉSENTATION DE L'APPLICATION

Exemple de sortie

- Fichier result.out
- dizziness: 1
- headache: 2
- rash: 2

PROBLÈMES IDENTIFIÉS DANS LE CODE INITIAL

- ✓ Code peu modulaire
 - ✓ Commentaires inutiles
 - ✓ Responsabilités mélangées dans la même class
 - ✓ Code difficile à maintenir et tester
- 
- Several white diagonal lines of varying thicknesses are drawn across the right side of the slide, extending from the top right towards the bottom left.

SOLUTIONS MISES EN PLACE

- ✓ Refactorisation du code (try-with-resources – naming camelCase – indenter les calss etc...)
- ✓ Séparation des responsabilités
- ✓ Création d'interfaces et des class
- ✓ Amélioration de la structure du projet
- ✓ Mise en place de la documentation avec Javadoc

ARCHITECTURE MISE EN PLACE

- ✓ **ISymptomReader** : interface de lecture
- ✓ **ISymptomWriter** : interface d'écriture
- ✓ **ReadSymptomDataFromFile** : lecture fichier
- ✓ **WriteSymptomDataToFile** : écriture fichier
- ✓ **AnalyticsCounter** : logique métier
- ✓ **Main** : déroulé global des étapes

PRINCIPE D'INJECTION DE DÉPENDANCES

- ✓ **AnalyticsCounter reçoit les dépendances via son constructeur**
- ✓ **Permet de remplacer facilement les sources de données**
- ✓ **Améliore la flexibilité et la testabilité**

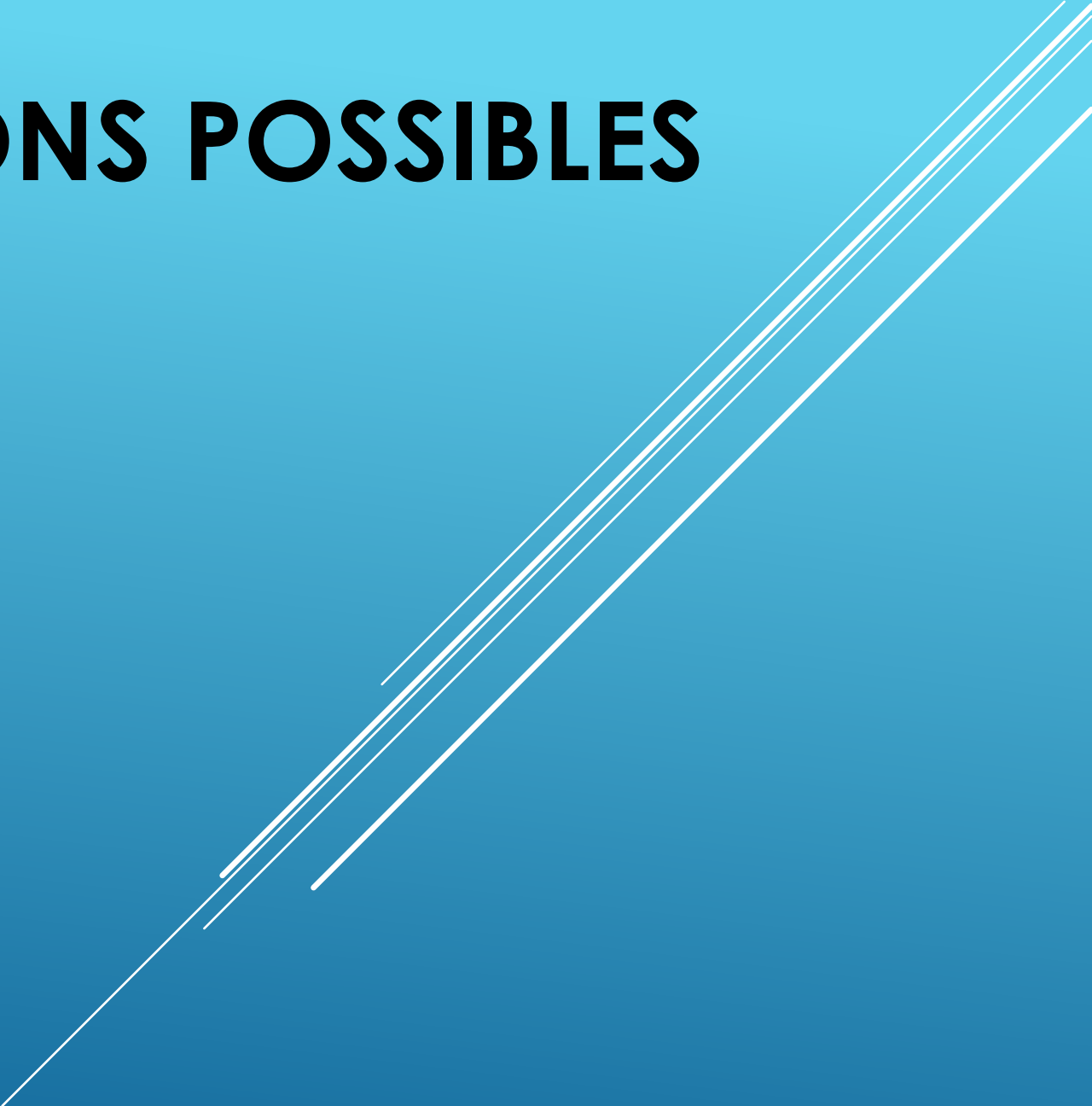
DIFFICULTÉS RENCONTRÉES

- ✓ Comprendre l'architecture du projet
- ✓ Identifier les responsabilités des classes

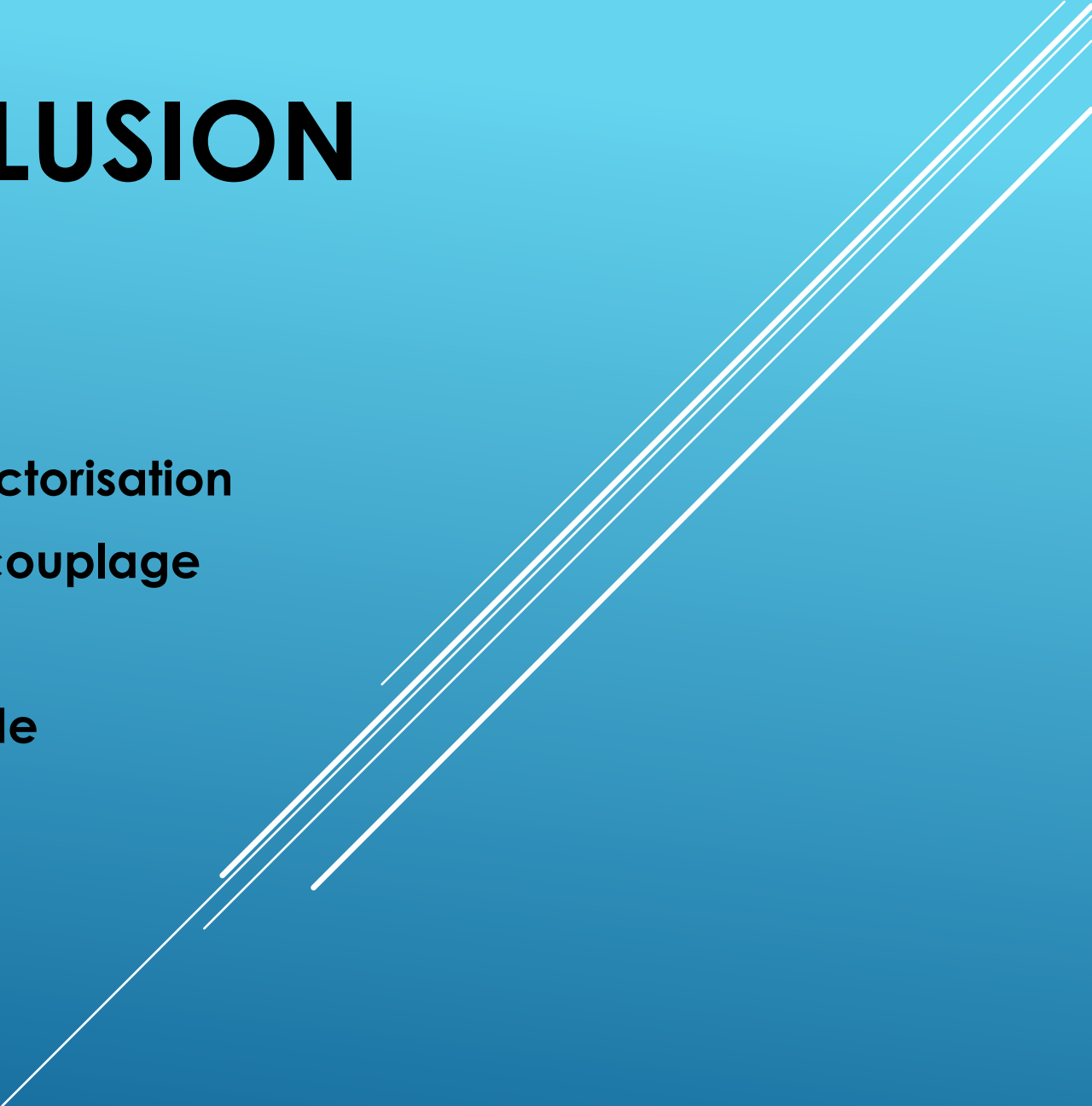
SOLUTIONS APPORTÉES

- ✓ Analyse du code existant
- ✓ Refactorisation progressive
- ✓ Utilisation de Map pour compter les occurrences
- ✓ Utilisation de TreeMap pour trier les résultats

AMÉLIORATIONS POSSIBLES

- ✓ Ajout de tests unitaires (JUnit)
 - ✓ Utilisation d'un système de logging
 - ✓ Gestion d'erreurs plus avancée
- 
- Several thin, parallel white lines of varying lengths and slopes are positioned in the lower right quadrant of the slide, extending from the bottom right towards the center.

CONCLUSION

- ✓ **Projet permettant de comprendre la refactorisation**
 - ✓ **Utilisation des interfaces pour réduire le couplage**
 - ✓ **Application des principes de conception**
 - ✓ **Amélioration de la maintenabilité du code**
- 
- Several thin, parallel white lines are drawn diagonally across the bottom right portion of the slide, starting from the bottom edge and extending towards the top right corner.